

Class 6: R Functions

Steven Studdard (PID: A19123470)

Table of contents

Background	1
--------------------------------------	---

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

Q1a+b+c

Here we will create a function that adds some numbers. Let's call it `add()`.

Input arguments can be either “**required**” or “**optional**”. The latter will have fall-back default values that will be used if the user does not specify them.

```
#Creating a basic function
add <- function(x, y=0) {
  x + y
}

#Having the add function work with multiple arguments)
add2 <- function(x=c(a,b,c), y=0){
  sum(x) + y
}

#Having the add function work with any amount of arguments.
add3 <- function(...){
```

```
    sum(...)  
}
```

Can we use our new function:

```
add3(4, 7)
```

```
[1] 11
```

```
add3(c(4,7,10))
```

```
[1] 21
```

```
add3(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

```
add4 <- function(x, y=0, z=0){  
  return(sum(x,y,z))  
  cat("Is it break time yet\n")  
}  
  
add4(10,100)
```

```
[1] 110
```

A generate_dna() function

A useful function here is the “base R” **sample()** function:

```
sample(1:5, size=10, replace = TRUE)
```

```
[1] 3 2 4 4 1 3 4 1 5 2
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
sample(x=c("A","C","G","T"), size = 10, replace = TRUE)
```

```
[1] "G" "C" "A" "A" "T" "T" "A" "T" "A" "C"
```

Q2a: Write a `generate_dna()` function.

```
#Writing a basic dna function and running it.
generate_dna <- function(len){
  sample(x=c("A","C","G","T"), size = len, replace = TRUE)
}
generate_dna(10)
```

```
[1] "A" "C" "T" "G" "C" "G" "A" "C" "C" "T"
```

Q2b:

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`.

```
#Generate DNA function
generate_dna <- function(len, single.element = TRUE){
  bp <- sample(x=c("A","C","G","T"), size = len, replace = TRUE)
  #Adding an if statement for an optional single string or single character output
  if(single.element){
    bp <- paste(bp, collapse="")
  }

  return(bp)
}

generate_dna(10, FALSE)
```

```
[1] "A" "G" "G" "G" "A" "C" "C" "G" "A" "C"
```

```
generate_dna(10, TRUE)
```

```
[1] "AGTTCCTTTC"
```

Q2c:

```

generate_dna <- function(len, single.element = TRUE){
  #Creating the first line of the sequence like a header.
  cat(">len",len, sep="")
  bp <- sample(x=c("A","C","G","T"), size = len, replace = TRUE)
  #Same as in Q2b
  if(single.element){
    bp <- paste(bp, collapse="")
  }
  #Printing out the function output for the character on a separate line using "\n"
  cat("\n", bp, sep = "")
}

generate_dna(44)

```

```

>len44
TCATCCTATCGCTTCATCTGCGCAATTGCCCTATATGTGGGTAT

```

Q3

```

#Same thing as the generate function but with protein instead of nucleotides
generate_protein <- function(len, single.element = TRUE){

  cat(">len",len, sep="")
  aa <- sample(x=c("A", "R", "N", "D", "C", "E",
                  "Q", "G", "H", "I", "L", "K",
                  "M", "F", "P", "S", "T", "W",
                  "Y", "V"), size = len, replace = TRUE)

  if(single.element){
    aa <- paste(aa, collapse="")
  }

  cat("\n", aa, sep = "")
}

generate_protein(20)

```

```

>len20
PMSNYGRRWFHWEQDGCDAQ

```

Q4

```
#A for loop running a function of x:y lengths, then printing it out with a line break every 1
for(l in 6:13){
  cat(generate_protein(l), "\n")
}
```

```
>len6
AFHTCI
>len7
CFHDYSE
>len8
DNWREKHL
>len9
WRNAHNCPY
>len10
QCQNVHRCVC
>len11
VLCYWVWIQWW
>len12
GPYTMNCPYWEE
>len13
KHNPISDHYMTMS
```

Q5

Length	Best Hit % Coverage	Best Hit % Identity	Unique? (Y/N)
6	100%	100%	N
7	100%	100%	N
8	88%	100%	Y
9	89%	100%	Y
10	80%	100%	Y
11	100%	81.82%	Y
12	75%	88.89%	Y
13	85%	81.82%	Y

- From 8 and on my sequences begin to look unique in nature.
- I don't think it is very probable to have exact sequences as 20^n where n is length is quite large. When we get to these larger numbers. For example at 20^8 we are already looking at 25 billion potential combinations.

Q6

MHC Class 2 peptide presentation is usually around $\sim < 9$ amino acids long. Based on what we said earlier, there is a significantly higher probability of finding non-unique sequences the shorter our peptide is. These small sequences can appear by chance anywhere rather than larger ones, so if they were presented to our immune system and it acts on it. It could cause degradation of things that it isn't supposed to be attacking.